Governance, Risk, and Compliance Lab Guide

Cybersecurity Masterclass by Joshua Moses | CyberTrace Foundation · Oct 23, 2026 · @Joshua Moses

Lab overview

Governance, Risk, and Compliance is the discipline that usually gets taught as slides and paperwork. This lab makes it hands-on. By the end you will have stood up a real GRC platform, written governance artifacts, built a risk register, mapped controls to a framework, run an automated compliance scan that produces genuine audit evidence, tracked remediation, and measured the program with KPIs. You will run the full GRC operating lifecycle on one scenario, and you will finish with the kind of evidence package an auditor actually asks for.

The scenario: you are the GRC analyst for a mid-sized company. A Linux server supports a financially significant application, so it falls in scope for audit. Leadership wants to know whether it meets a recognized security baseline, what the risks are, and whether you can prove the controls work. Everything you build answers that, which is exactly what a GRC program is for.

Who this is for. Students who have finished the virtualization lab (or who can already build and run a Linux VM) and are comfortable at a command line. No prior GRC or audit experience is needed. Every command and every governance artifact is spelled out, with templates in the Appendix.

Time. Plan on 6 to 9 hours total. The build (Parts A through C) takes 2 to 3 hours and only has to be done once. The six exercises take 4 to 6 hours and can be split across sessions, since the platform persists.

What you need.

- A host machine with at least 12 GB RAM and 80 GB free disk. The GRC platform is light; the scan target is a small Linux VM.
- A hypervisor you already know (VMware Workstation, VirtualBox, Fusion, or UTM).
- Internet access from the lab VM during the build, to pull containers and SCAP content.

A note on what GRC is, before you start. GRC is not three separate jobs. Governance decides what must be achieved and who is accountable. Risk management decides what to prioritize. Compliance verifies the required controls actually operate. The single most

important idea in this whole lab, and the one the last exercise drives home: passing an audit is not the same as being secure, and being secure is not the same as passing an audit. A good GRC program makes those two things line up. Keep that in mind at every step.

Ground rules.

1. Everything runs in your own lab VMs. The compliance scan and any remediation you apply are run only against your own lab systems.
2. Remediation scripts that a scanner generates can change system configuration. Always review them before running, and always run them against a snapshot you can roll back, never a system you care about.
3. The governance artifacts you write (policies, risk register, audit evidence) are realistic examples for learning. Adapt them to a real framework and real obligations before using anything like them in production.
4. Snapshot every VM at the clean build stage before you start the exercises, so you can roll back and rerun.

Principles of GRC

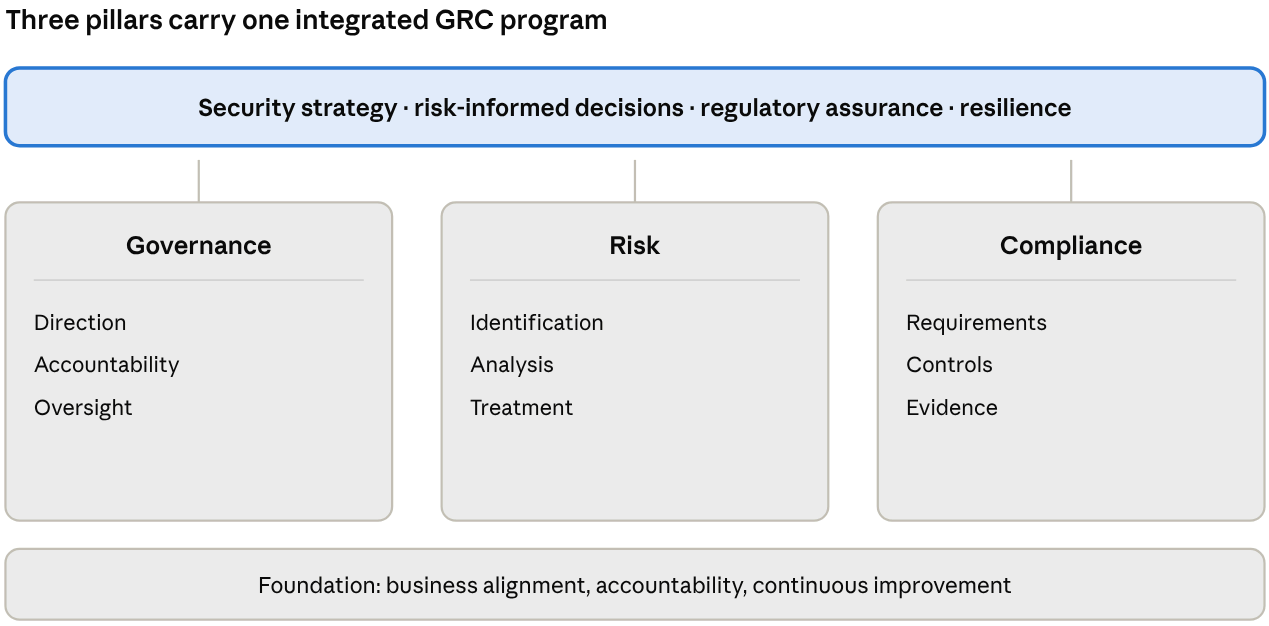
Governance, Risk, and Compliance is an integrated framework for running an organization securely: managing uncertainty, meeting regulatory obligations, and keeping cybersecurity aligned with business objectives. It rests on three disciplines. **Governance** establishes how security decisions get made, who is accountable, and how security supports the business. **Risk management** identifies, evaluates, prioritizes, and manages the threats that could hurt the organization. **Compliance** ensures the organization meets its laws, regulations, contracts, and internal policies, and can prove it.

The 15 core principles. These are the ideas the exercises put into practice. They are synthesized from established governance and risk practice rather than one standardized list, but every mature program reflects them.

#	Principle	What it means
1	Accountability	Assign clear ownership for decisions, controls, and risk
2	Transparency	Give leadership accurate visibility into posture, risk, and obligations

#	Principle	What it means
3	Business alignment	Tie security initiatives to business objectives and strategy
4	Risk-based decision making	Spend resources by risk exposure, impact, and priority
5	Regulatory compliance	Identify and meet applicable legal, regulatory, and contractual requirements
6	Policy management	Establish, communicate, enforce, and review policies and standards
7	Internal controls	Use preventive, detective, and corrective controls to manage risk
8	Continuous monitoring	Continuously evaluate risk, control effectiveness, and compliance
9	Defense in depth	Layer complementary controls to cut likelihood and impact
10	Least privilege	Grant only the access users, systems, and apps actually need
11	Segregation of duties	Separate conflicting responsibilities to reduce fraud and misuse
12	Auditability	Keep reliable records and evidence that controls operate
13	Continuous improvement	Use assessments, audits, incidents, and metrics to improve
14	Third-party risk management	Evaluate and monitor vendor and partner risk
15	Resilience	Prepare to withstand, respond to, and recover from disruption

The three pillars. Governance, Risk, and Compliance are distinct but interlocking. Governance sets direction, accountability, and oversight. Risk handles identification, analysis, and treatment. Compliance manages requirements, controls, and evidence. Together they form one integrated program that delivers security strategy, risk-informed decisions, regulatory assurance, and business resilience.



three pillars of GRC

Pillar 1, Governance sets the structures, responsibilities, policies, and oversight to manage security effectively: executive accountability, defined roles, policy enforcement, strategic alignment, and performance measurement. Example: a CISO establishes an identity security policy requiring privileged access to be approved, monitored, and periodically reviewed.

Pillar 2, Risk Management identifies threats and vulnerabilities, evaluates their consequences, and chooses responses. The process runs: identify critical assets and processes, identify threats and vulnerabilities, assess likelihood and impact, determine inherent risk, evaluate existing controls, determine residual risk, select and implement treatments, then monitor and report. The four treatment strategies:

Strategy	Example
Mitigate	Deploy MFA to reduce credential-compromise risk
Accept	Formally accept a low risk within approved tolerance
Transfer / share	Use insurance or contracts to share the financial consequences
Avoid	Discontinue an activity that introduces unacceptable risk

Pillar 3, Compliance establishes whether the organization meets its requirements and can prove its controls work. Key activities: maintain a compliance obligations register, map controls to requirements, collect audit evidence, find deficiencies, track remediation, and document exceptions. Example: an organization subject to SOX must show that privileged access to financially significant systems is authorized, monitored, and reviewed.

Major GRC frameworks you should know, and which this lab touches:

Framework	Primary purpose
NIST CSF 2.0	Organizes cyber risk into Govern, Identify, Protect, Detect, Respond, Recover
NIST RMF	Structured process for managing security and privacy risk
NIST SP 800-53	Catalog of security and privacy controls
ISO/IEC 27001	Requirements for an information security management system (ISMS)
COBIT 2019	Governance and management of enterprise IT
CIS Controls	Prioritized cybersecurity safeguards (what you will scan against)
SOC 2	Independent attestation against Trust Services Criteria

Regulatory obligations such as HIPAA, GDPR, PCI DSS, and SOX also shape a program depending on the organization.

GRC applied to IAM. Identity security is one of the most important technical areas any GRC program governs, and it is where governance, risk, and compliance meet most concretely. The control areas and how they show up:

GRC control area	Example implementation
Access governance	Formal access request and approval workflows
Privileged access management	PAM tooling, session monitoring, credential management

GRC control area	Example implementation
Least privilege	Role-based access, just-in-time access, zero standing privilege
Segregation of duties	Identify and prevent conflicting entitlements
Access certification	Periodic reviews of user and privileged access
Joiner-Mover-Leaver	Automated provisioning and deprovisioning
Service account governance	Ownership, inventory, credential rotation, lifecycle
Audit evidence	Access logs, approvals, certification results, privileged activity
Third-party access	Time-limited vendor access with continuous oversight

An auditor may ask for evidence that every privileged account has an owner, that access was approved, and that privileged activity is monitored. A working GRC program makes that evidence available and verifiable, on demand.

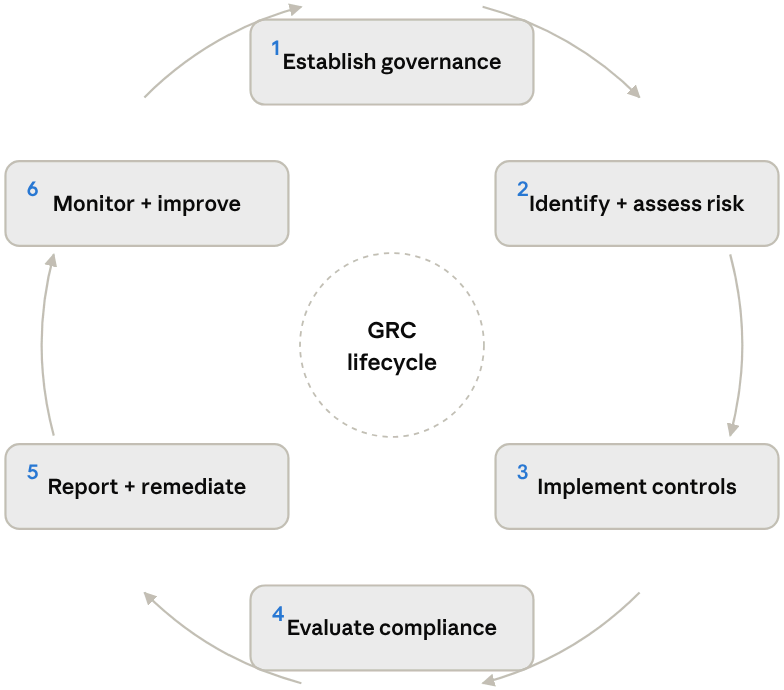
Measuring effectiveness. A mature program uses measurable indicators, not just the existence of policies. The KPIs you will use in the final exercise: control effectiveness rate, risk remediation time, audit finding closure rate, policy exception count, access certification completion, privileged account coverage, third-party assessment coverage, and repeat audit findings.

The most important principle. Compliance does not automatically equal security, and security does not automatically equal compliance. An organization can pass an audit while still carrying exploitable vulnerabilities, excessive privileges, or a weak incident response. Effective GRC integrates all three disciplines: governance determines what must be achieved, risk management determines what to prioritize, and compliance verifies that obligations are met and controls operate. This lab is built so you feel that gap directly: you will pass part of a compliance scan while real risk remains, and the program is what closes it.

The GRC operating lifecycle. The program runs as a continuous loop. The six exercises in this lab are the six stages of that loop, in order.

The GRC program runs as a continuous loop

The six exercises in this lab are these six stages, in order

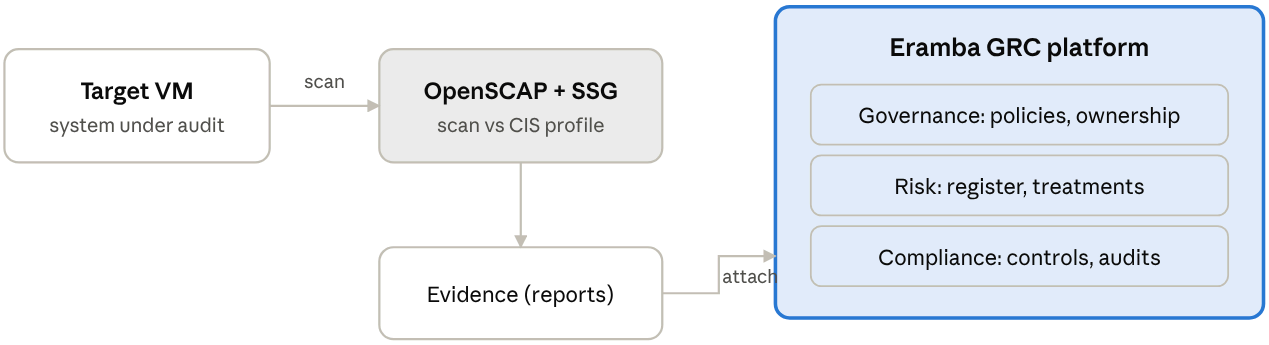


GRC operating lifecycle · 6 stages

Lab architecture and software

The lab is two VMs. One runs the GRC platform where you manage governance, risk, and compliance. The other is the target system you assess and collect audit evidence from. This mirrors reality: GRC tooling is separate from the systems it governs.

Scans produce evidence; the platform governs the program



lab data flow · scan to platform

Component	Role	OS	Specs
GRC VM	Eramba GRC platform (Docker)	Ubuntu Server 22.04 or 24.04	2 vCPU, 4 GB RAM, 40 GB disk
Target VM	System under audit, scanned with OpenSCAP	Ubuntu Server 22.04 or 24.04	2 vCPU, 2 GB RAM, 25 GB disk

You can run both on one host comfortably. If disk is tight, you can even install OpenSCAP directly on the GRC VM and scan it as its own target, but keeping them separate teaches the real pattern.

The software, and why each tool is here.

- **Eramba Community Edition** is your GRC platform. It is where the three disciplines live in one place: you write and version security policies (governance), build and score a risk register with treatments (risk), and maintain a control catalog mapped to frameworks with audit records and evidence (compliance). It is the tool that turns GRC from scattered spreadsheets into a managed program with ownership, review dates, and an audit trail. Free and open source, and genuinely used by real GRC teams.
- **OpenSCAP** (the `oscap` scanner) is your automated compliance assessment engine. It evaluates a system against a security baseline and produces a machine- and human-readable report of exactly which controls pass and fail. This is how compliance stops being a checkbox and becomes evidence: a timestamped, repeatable scan result.

- **SCAP Security Guide (SSG)** is the content OpenSCAP scans against. It packages recognized baselines (CIS Benchmarks, DISA STIG) into profiles you point the scanner at. You will scan against a CIS profile, which is one of the frameworks listed in the principles section.
- **The target VM** is a plain Ubuntu server standing in for the financially significant system in your scenario. You scan it, find gaps, remediate, and rescan, which is the compliance evaluation loop in miniature.

Everything is free and open source. No license or account is required. The audit evidence you generate (SCAP reports) gets attached to control records in Eramba, which ties the two tools into one GRC workflow.

Part A: build the lab VMs and install Docker

Two VMs: the GRC VM (which needs Docker) and the Target VM (a plain Ubuntu server). If you did the virtualization lab, the VM creation is familiar.

A1. Build the GRC VM. Create a VM with 2 vCPU, 4 GB RAM, 40 GB disk and install Ubuntu Server 22.04 or 24.04. Give it internet access. Update it:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg git
```

A2. Install Docker Engine and Compose on the GRC VM.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

A3. Let your user run Docker without sudo.

```
sudo usermod -aG docker $USER
newgrp docker
docker --version
docker compose version
```

Both version commands should print cleanly. Docker 27+ and Compose v2 are what this lab expects.

A4. Build the Target VM. Create a VM with 2 vCPU, 2 GB RAM, 25 GB disk and install Ubuntu Server 22.04 or 24.04. This is the system you will audit. Do nothing special to it; a default install is the point, because a default install has gaps for you to find. Update it:

```
sudo apt update && sudo apt upgrade -y
```

A5. Note both IPs. Run `ip a` on each VM and record the addresses. You reach Eramba's web UI from your host browser at the GRC VM's IP. For the rest of the guide, `GRC_IP` and `TARGET_IP` mean those addresses.

That is the foundation. Next you deploy the GRC platform and the scanner.

Part B: deploy and configure Eramba (the GRC platform)

Eramba Community Edition ships as a Docker Compose stack (the app plus a MariaDB database). The official repo has a ready-to-use compose file. All of this runs on the GRC VM. Reference: Eramba Docker installation.

B1. Clone the Eramba Docker repo.

```
cd ~ && git clone https://github.com/eramba/docker.git eramba
cd eramba
```

B2. Review the compose file. The community deployment uses `docker-compose.simple-install.yml`. It defines the Eramba app, the database, and an Apache front end. Open it and confirm the exposed port (HTTPS is preconfigured). You do not need to change anything for a lab, but it is worth reading once to see the moving parts.

B3. Start the stack. The first run pulls the images and initializes the database, which takes several minutes.

```
docker compose -f docker-compose.simple-install.yml up -d
```

B4. Watch it initialize.

```
docker compose -f docker-compose.simple-install.yml ps
docker compose -f docker-compose.simple-install.yml logs -f
```

Wait until the app container reports it is ready and stops restarting. The database setup on first boot is the slow part; be patient.

B5. Complete the web installer. From your host browser, go to `https://GRC_IP` (HTTPS, self-signed certificate, accept the warning). Eramba's first-run screen walks you through the final setup: accept the license terms for the community edition, confirm the system health checks pass, and set your admin account. If a health check flags a missing cron job or setting, note it; the installer tells you the fix.

B6. Log in and look around. Log in with the admin account you just set. Spend a few minutes in the main modules, because these map directly to the three pillars and you will use each one:

- **Governance:** Policies, where you store and version security policies with owners and review dates.
- **Risk Management:** the risk register, where risks get scored, owned, and assigned a treatment.
- **Compliance Management:** controls and the control catalog, plus audits and the ability to attach evidence.

B7. Set up a cron runner (recommended). Eramba uses scheduled jobs for reviews, notifications, and reminders, which are part of what makes it a living program rather than a static database. The repo documents the cron container or command; enable it so review dates and notifications actually fire. For a short lab you can skip this, but turning it on shows how continuous monitoring (principle 8) works in practice.

If the app container restarts in a loop or the installer will not complete, see Troubleshooting. A fresh database sometimes needs a second start.

Part C: install OpenSCAP and the SCAP Security Guide

This runs on the Target VM, the system you will audit. OpenSCAP is the scanner; the SCAP Security Guide is the baseline content it scans against. Reference: OpenSCAP.

C1. Install the scanner and content.

```
sudo apt update
sudo apt install -y openscap-scanner openscap-utils ssg-base ssg-debderived
oscap --version
```

The `oscap --version` output confirms the scanner is installed. On some Ubuntu versions the content package is named `scap-security-guide`; if the `ssg-*` packages are not found, try `sudo apt install -y scap-security-guide`.

C2. Find the content file for your OS. The SSG content lives under `/usr/share/xml/scap/ssg/content/`. List it and pick the data stream that matches your Ubuntu version:

```
ls /usr/share/xml/scap/ssg/content/
```

You are looking for a file like `ssg-ubuntu2204-ds.xml` (22.04) or `ssg-ubuntu2404-ds.xml` (24.04). For the rest of this guide, `SSG_FILE` means that path.

C3. See which profiles are available. A profile is a named baseline (CIS Level 1, CIS Level 2, STIG, and so on). List them:

```
oscap info SSG_FILE
```

Note the profile IDs. You will use the CIS Level 1 Server profile, whose ID looks like `xccdf_org.ssgproject.content_profile_cis_level1_server`. For the rest of the guide, `PROFILE` means that ID.

C4. Run a baseline scan now. This is your before picture. You will keep it to compare against the after scan in Exercise 4.

```
sudo mkdir -p ~/scap
sudo oscap xccdf eval \
  --profile PROFILE \
  --results ~/scap/baseline-results.xml \
  --report ~/scap/baseline-report.html \
  SSG_FILE
```

The scan prints each rule as pass or fail and writes an HTML report. Open `~/scap/baseline-report.html` in a browser (copy it to your host, or serve it). It shows your score and every finding. A default Ubuntu install typically fails a lot of CIS rules; that is expected and is exactly the raw material for the exercises.

C5. Snapshot the clean build. With Eramba running on the GRC VM, OpenSCAP installed and a baseline scan captured on the Target VM, power both down and snapshot each one, named `clean-build`. You will roll the Target VM back to this snapshot before applying remediation, so you can rerun the before/after cleanly.

The build is done. Everything from here is GRC work.

Exercise 1: Establish governance

Stage goal: define policies, ownership, accountability, risk appetite, and oversight. Governance comes first because it decides what the whole program is trying to achieve and who answers for it. Principles in play: accountability (1), transparency (2), business alignment (3), policy management (6).

What you will do

1. **Write a security policy in Eramba.** Using the policy template in the Appendix, create one real policy: an Access Control Policy that requires privileged access to be approved, monitored, and periodically reviewed. In Eramba's Governance/Policies module, create the policy, assign an owner, and set a review date. A policy with no owner and no review date is not governance, it is a document.
2. **Define accountability.** For your scenario, name who owns what: who owns the policy, who owns the risks you are about to find, who owns the target system. Record these as owners in Eramba. Accountability only works when it is assigned to a person, not a team in the abstract.
3. **Set your risk appetite.** Write one or two sentences stating how much risk the organization will tolerate, for example: "We will not accept any high risk to financially significant systems without CISO sign-off." This single statement is what makes risk-based decisions possible later; without it, every risk looks equally urgent.
4. **State the oversight mechanism.** Note how leadership will get visibility: a monthly risk report, a quarterly access review, a dashboard. This is principle 2, transparency, made concrete.
5. **Tie it to the business.** In one line, connect this program to a business objective: protecting the financially significant system supports accurate financial reporting and audit readiness. Governance that is not tied to a business objective gets defunded.

Intended result and why. You leave this stage with a written, owned Access Control Policy in Eramba, named accountability for policy/risk/system, a stated risk appetite, an oversight mechanism, and a business-alignment statement. This matters because

governance is the control surface for everything that follows. The risk appetite you set here decides which risks you must treat in Exercise 2; the ownership you assign decides who closes the findings in Exercise 5. Teams that skip governance end up doing compliance theater: lots of activity, no accountability, no clear goal.

Deliverable: one owned policy in Eramba with a review date, an accountability list, a risk-appetite statement, and an oversight plan.

Exercise 2: Identify and assess risks

Stage goal: evaluate assets, threats, vulnerabilities, and impact, and build a scored risk register. Principles in play: risk-based decision making (4), internal controls (7), defense in depth (9).

Work the risk process from the principles section, using the risk register template in the Appendix and recording everything in Eramba's Risk Management module.

What you will do

1. **Identify critical assets.** Your scenario centers on the financially significant Target system. List it and anything it depends on (the data it holds, the accounts that access it). You cannot assess risk without knowing what you are protecting.
2. **Identify threats and vulnerabilities.** For the Target system, list plausible threats (credential compromise, privilege misuse, unpatched exploitation) and the vulnerabilities that enable them. Your baseline SCAP scan from Part C is a real source here: every failed CIS rule is a candidate vulnerability.
3. **Build at least four risks in the register.** For each, write a clear risk statement (threat + vulnerability + consequence), for example: "An attacker who phishes a privileged credential could access the financial system because MFA is not enforced, leading to fraud and audit failure." Enter each into Eramba as a risk.
4. **Score inherent risk.** For each risk, rate likelihood and impact on your scale (Eramba supports this directly) and record the inherent risk, which is the risk before controls. This is what makes the register prioritizable.
5. **Evaluate existing controls and determine residual risk.** For each risk, note what control (if any) already reduces it, and record the residual risk, which is what remains after existing controls. The gap between inherent and residual is the value your controls provide.

6. **Select a treatment for each.** Using the four strategies (mitigate, accept, transfer, avoid), choose and record a treatment for every risk, checked against the risk appetite you set in Exercise 1. A high risk to the financial system cannot simply be accepted if your appetite forbids it; that constraint is governance driving risk, exactly as intended.

Intended result and why. You leave this stage with a populated risk register in Eramba: four or more risks, each with a statement, owner, inherent and residual scores, existing controls, and a chosen treatment consistent with your risk appetite. This matters because the register is how an organization decides where to spend limited security resources. Without scored, owned risks, prioritization is just opinion. With them, you can say exactly which risk to treat first and why, which is what risk-based decision making means.

Deliverable: a risk register in Eramba with four or more scored, owned, treated risks.

Exercise 3: Implement controls

Stage goal: apply technical, administrative, and physical safeguards, and map them to a framework so you can prove coverage. Principles in play: internal controls (7), least privilege (10), segregation of duties (11), auditability (12).

What you will do

1. **Define controls for your treatments.** For each risk you chose to mitigate in Exercise 2, define the specific control that does it. Example: for the MFA risk, the control is "MFA enforced on all privileged access." Enter each control in Eramba's Compliance/Controls module with an owner and a description of how it operates.
2. **Map controls to a framework.** This is the step that makes compliance provable. Map each control to the framework requirement it satisfies. Use the CIS Controls (which your SCAP scan tests against) or NIST SP 800-53. Eramba lets you link controls to requirements; some frameworks are importable as packages. The control mapping template in the Appendix gives you the structure. Mapping is what lets you answer "are we compliant with requirement X?" with evidence instead of a guess.
3. **Classify each control.** Mark each as preventive, detective, or corrective. A healthy program has all three; if every control you listed is preventive, you have no way to catch what gets through, which is the point of defense in depth.
4. **Apply the IAM controls concretely.** Since identity is the most-governed technical area, make at least two of your controls IAM controls: least privilege (remove unneeded admin rights on the Target system) and auditability (ensure privileged

actions are logged). These are controls you can actually implement and later show evidence for.

5. **Note what is not yet implemented.** Be honest about which controls exist today versus which are planned. A control you claim but have not implemented is the exact gap that turns into an audit finding. Recording the real state now is what makes Exercise 4 meaningful.

Intended result and why. You leave this stage with a set of controls in Eramba, each owned, classified as preventive/detective/corrective, and mapped to a framework requirement, with an honest status. This matters because controls are where risk treatment becomes real, and the framework mapping is what makes the program auditable. An auditor does not ask "do you feel secure?"; they ask "show me the control that satisfies this requirement, and prove it works." This exercise builds the first half of that answer; the next exercise builds the proof.

Deliverable: a control catalog in Eramba with owned, classified, framework-mapped controls and honest implementation status.

Exercise 4: Evaluate compliance

Stage goal: map controls to requirements, test whether they actually operate, and collect audit evidence. This is where OpenSCAP turns compliance from a claim into proof. Principles in play: continuous monitoring (8), auditability (12), regulatory compliance (5).

What you will do

1. **Review the baseline scan.** Open the `baseline-report.html` from Part C. This is your as-is compliance state against the CIS profile: the overall score and every pass and fail. Each failed rule is a control that is required but not operating. Note your baseline score.
2. **Map scan findings to your controls.** Cross-reference the SCAP failures against the controls you defined in Exercise 3. Some failures map to controls you claimed were implemented, which means they are not actually operating. That gap, a claimed control that fails its test, is the single most important thing an audit finds. Write these down.
3. **Collect the evidence.** The SCAP HTML and XML reports are audit evidence: timestamped, repeatable, independently verifiable. In Eramba, attach the baseline report to the relevant control records as evidence. This is auditability (principle 12) made literal: you are not saying the control works, you are attaching proof of its state.

4. **Experience the key principle directly.** Look at your score. You may be passing a majority of rules while real risk remains (for example, logging passes but MFA is absent). You can be partially compliant and still insecure. Write one sentence naming a place where your scan looks acceptable but genuine risk remains. This is the lesson the whole lab is built around.
5. **Identify the deficiencies and exceptions.** List every control that failed its test. For any failure you will not fix (a rule that does not apply to your environment), document a formal exception with a reason and an approver, rather than silently ignoring it. Documented exceptions are how mature programs stay honest; undocumented ones are how they fail audits.

Intended result and why. You leave this stage with a scored compliance assessment, scan findings mapped to your controls, evidence attached in Eramba, a list of deficiencies, documented exceptions, and a concrete example of the compliance-is-not-security gap. This matters because evaluation is the moment a GRC program tells the truth. Everything before it was intention; the scan is measurement. The habit of testing controls and keeping the evidence, rather than assuming controls work, is the difference between a program that passes a real audit and one that is surprised by it.

Deliverable: a scored SCAP assessment, findings mapped to controls, evidence attached in Eramba, a deficiency list, and documented exceptions.

Exercise 5: Report and remediate

Stage goal: communicate findings, assign corrective actions, and track risk decisions to closure. Finding problems is only half the job; GRC exists to get them fixed and to prove the fix. Principles in play: transparency (2), continuous improvement (13), resilience (15).

What you will do

1. **Remediate on the Target VM, safely.** OpenSCAP can generate a remediation script from your scan results. Generate it, review it (this is required, not optional, since it changes system config), then run it against your snapshot:

```
sudo oscap xccdf generate fix \
  --profile PROFILE \
  --fix-type bash \
  --output ~/scap/remediation.sh \
  SSG_FILE
less ~/scap/remediation.sh
sudo bash ~/scap/remediation.sh 2>&1 | tee ~/scap/remediation.log
```

Reviewing before running is a control in itself: you are the change-approval step.

2. **Rescan to prove the fix.** Run the scan again and compare the score to your baseline:

```
sudo oscap xccdf eval \
  --profile PROFILE \
  --results ~/scap/after-results.xml \
  --report ~/scap/after-report.html \
  SSG_FILE
```

The after score should be meaningfully higher. That before/after pair is your evidence that remediation worked, which is far stronger than claiming it did.

3. **Write the reports for two audiences.** Produce a short management summary (what the risk was, what you fixed, residual risk, in plain language) and a technical findings list (specific failed rules, the fix applied, verification). The same work, two audiences, is the transparency principle in practice. Attach both in Eramba.
4. **Assign corrective actions for what remains.** Not everything gets fixed by one script. For each remaining deficiency, create a corrective action in Eramba with an owner and a due date, linked to the risk it addresses. An owned, dated action is the difference between a finding that closes and one that recurs.
5. **Update the risk register.** Now that controls are improved, update the residual risk scores on the affected register entries. Watching residual risk drop as controls take effect is the whole point of the register, and it shows leadership the program is working.

Intended result and why. You leave this stage with a before/after SCAP comparison proving measurable improvement, a management report and a technical report attached in Eramba, owned and dated corrective actions for remaining gaps, and an updated risk register. This matters because reporting and remediation are where GRC delivers value instead of just documenting problems. The before/after evidence is what you show leadership and auditors; the corrective-action tracking is what keeps findings from becoming repeat findings next cycle.

Deliverable: a before/after scan comparison, two audience reports in Eramba, owned corrective actions, and an updated risk register.

Exercise 6: Monitor and improve

Stage goal: reassess risk, review control performance, measure the program, and refine. This closes the loop and feeds the next cycle. Principles in play: continuous monitoring (8), continuous improvement (13), auditability (12).

What you will do

1. **Measure the program with KPIs.** Using the KPI template in the Appendix, compute the indicators you have data for after this cycle. At minimum: control effectiveness rate (percentage of tested controls passing, straight from your after-scan), audit finding closure rate (how many deficiencies you closed versus found), and privileged account coverage (how many privileged accounts are governed by your controls). You cannot improve what you do not measure, and these numbers are how you prove the program works to leadership.
2. **Set up continuous monitoring.** A one-time scan is a snapshot; monitoring is a program. Schedule the SCAP scan to run on a cadence (a cron job on the Target VM) so compliance drift is caught, not discovered at the next audit. If you enabled Eramba's cron runner in Part B, confirm that control review dates and reminders are firing. This is principle 8 operating for real.
3. **Reassess the risks.** Revisit the risk register. Did residual risk drop where you expected? Did remediation introduce anything new? Mark any risk whose score changed, and flag any new risk the cycle revealed. The register is a living document, not a one-time deliverable.
4. **Review control performance and repeat findings.** Compare this cycle to the baseline. Which controls held, which failed again? A repeat finding (a deficiency you saw last time and still see) is a red flag that remediation was cosmetic, and tracking it is how you catch that.
5. **Write the improvement note and feed it forward.** One short page: what the program achieved this cycle, where it fell short, and what changes next cycle (new controls, new policies, new PIRs for the risk team). This note seeds the next turn of the lifecycle, back at Establish Governance.

Intended result and why. You leave this stage with a KPI scorecard, scheduled continuous monitoring, a reassessed risk register, a control-performance review including any repeat findings, and a written improvement note. This matters because a GRC program that does

not measure and improve plateaus: it keeps passing the same checks while real risk accumulates. The teams that review KPIs and close the loop get measurably stronger each cycle. This exercise is also where the lab's central lesson lands fully: a mature program is not the one with the most policies, it is the one that measures whether its controls actually reduce risk, and improves when they do not.

Deliverable: a KPI scorecard, a monitoring schedule, an updated risk register, a control-performance review, and a one-page improvement note.

Expected results, deliverables, and grading

By the end a student should be able to show a running GRC platform populated with real governance, risk, and compliance artifacts, plus genuine audit evidence from a scan and a measurable before/after improvement. The table is both a student checklist and a grading rubric.

Stage	Deliverable	What "done" looks like
Establish governance	Policy + accountability + appetite	Owned Access Control Policy in Eramba with review date; named owners; risk-appetite statement; oversight plan
Identify + assess risk	Risk register	Four or more risks with statements, owners, inherent and residual scores, and treatments consistent with appetite
Implement controls	Control catalog	Owned controls, classified preventive/detective/corrective, mapped to a framework, with honest status
Evaluate compliance	Scored assessment + evidence	Baseline SCAP score, findings mapped to controls, evidence attached in Eramba, deficiencies and exceptions documented
Report + remediate	Before/after + reports	Measurable score improvement, management and technical reports, owned corrective actions, updated register
Monitor + improve	KPIs + improvement note	KPI scorecard, scheduled monitoring, reassessed register, repeat-finding review, improvement note

Grading suggestion (100 points). Each of the six stages is worth up to 14 points for completeness and correctness of its deliverable, for 84 total. The remaining 16 points: 8 for a correct, evidenced before/after compliance improvement (did remediation actually raise the score, and can the student prove it?), and 8 for the student's written explanation of the compliance-is-not-security gap they found in their own scan. Those two prove the student can both operate the tooling and think like a GRC professional rather than a checkbox-filler.

Minimum pass: a student passes when Eramba holds a real policy, a scored risk register, and framework-mapped controls; when they ran a SCAP scan, attached the evidence, and remediated with a measurable improvement; and when they can explain, in their own words, why passing the scan did not by itself make the system secure. As in real GRC, the judgment matters as much as the artifacts. A student who can populate Eramba but cannot explain why compliance and security are not the same thing has learned the tool, not the discipline.

Troubleshooting

Symptom	Likely cause	Fix
Eramba app container restarts in a loop	Database not ready on first boot	Wait a few minutes; if it persists, `docker compose -f docker-compose.simple-install.yml down` then `up -d` again
Eramba web installer will not complete	A failed system health check	Read the health check message; it names the fix (often a cron job or a PHP setting); apply it and reload
Cannot reach Eramba from the host	Wrong IP, port, or cert warning	Confirm `GRC_IP` with `ip a`; use `https://` and accept the self-signed cert
Eramba review dates and reminders never fire	Cron runner not enabled	Enable the cron container or command per the repo's docs (Part B step B7)
`oscap` content files not found	SSG content package not installed	`sudo apt install -y scap-security-guide` (or the `ssg-*` packages); confirm files exist under `/usr/share/xml/scap/ssg/content/`

Symptom	Likely cause	Fix
Scan errors with "profile not found"	Wrong profile ID or wrong content file	Run `oscap info SSG_FILE` and copy the exact profile ID; confirm the SSG file matches your Ubuntu version
Scan hangs or runs very long	A slow rule	Re-run with `--verbose` to see which rule; a full CIS scan can take several minutes, which is normal
Remediation script breaks something	Scripts change real config	That is why you review before running and run against a snapshot; roll back to `clean-build` and try again more selectively
HTML report will not open on the host	File is on the VM	Copy it over (`scp`), or serve it with `python3 -m http.server` from the report folder

If a VM gets into a bad state, that is what the `clean-build` snapshot is for. Roll back and redo the affected part. Rolling back the Target VM is also how you rerun the before/after remediation cleanly.

Appendix: templates

Copy these into your notes and into Eramba as you work.

Security policy template (Governance)

Field	Entry
Policy name	
Owner	
Purpose / scope	
Policy statements (what is required)	
Related framework requirement	
Effective date	
Review date	

Risk register template (Risk)

Field	Entry
Risk ID	
Risk statement (threat + vulnerability + consequence)	
Owner	
Affected asset	
Likelihood	
Impact	
Inherent risk	
Existing controls	
Residual risk	
Treatment (mitigate/accept/transfer/avoid)	

Control mapping template (Compliance)

Control ID	Control description	Type (prev/det/corr)	Owner	Framework requirement	Status	Evidence

Audit evidence log

Evidence item	Control it supports	Source (e.g. SCAP report)	Date collected	Where stored

KPI scorecard (Monitor)

KPI	How to compute it	This cycle
Control effectiveness rate	% of tested controls passing (from after-scan)	

KPI	How to compute it	This cycle
Risk remediation time	Avg time from finding to fix	
Audit finding closure rate	% of findings closed on time	
Policy exception count	Number and severity of approved exceptions	
Access certification completion	% of access reviews done on time	
Privileged account coverage	% of privileged accounts governed	
Third-party assessment coverage	% of in-scope vendors assessed	
Repeat audit findings	Count of findings seen in a prior cycle	

References

- Eramba (GRC platform) and Docker installation and the Eramba Docker repo
- OpenSCAP and the SCAP Security Guide
- CIS Controls and Benchmarks, Center for Internet Security
- NIST Cybersecurity Framework (CSF) 2.0
- NIST Risk Management Framework (RMF)
- NIST SP 800-53, Security and Privacy Controls
- ISO/IEC 27001
- COBIT 2019, ISACA

Software referenced in this guide: Eramba Community Edition (current release), OpenSCAP with the SCAP Security Guide. Check each project's page for the current version before you build; the commands change little, but always pull the latest stable release.

Cybersecurity Masterclass by Joshua Moses | CyberTrace Foundation.